

CONCURRENT SERVING BENCHMARK

Serving an LLM to 8 users at once — on a 16 GB Apple M5 laptop

vLLM (MLX backend) · 4-bit KV cache · energy measured with powermetrics

4 → 8

concurrent users

+28%

throughput @ 8 users

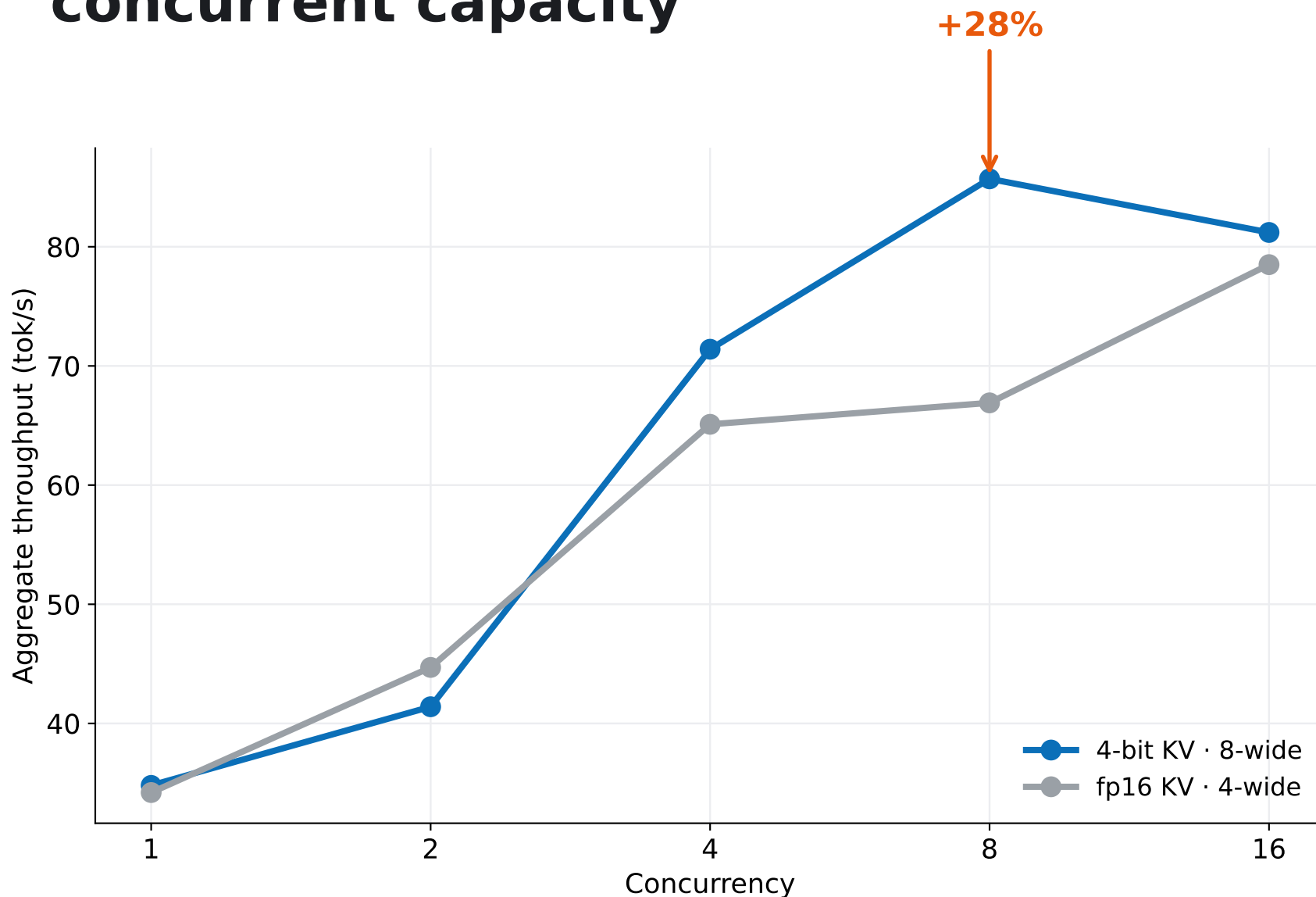
16.5s → 0.58s

first-token latency

~18 W

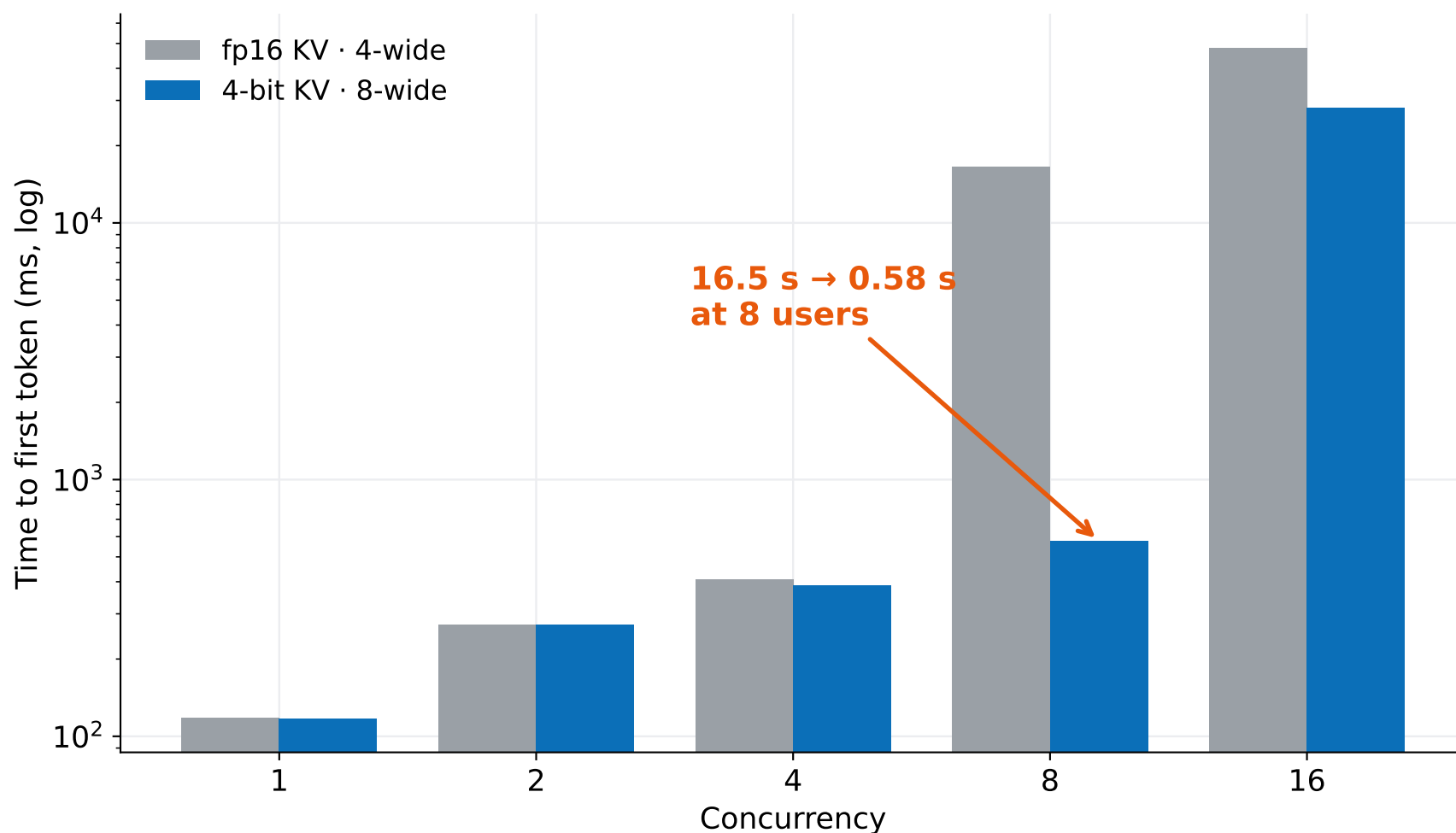
at full load

4-bit KV cache doubled concurrent capacity



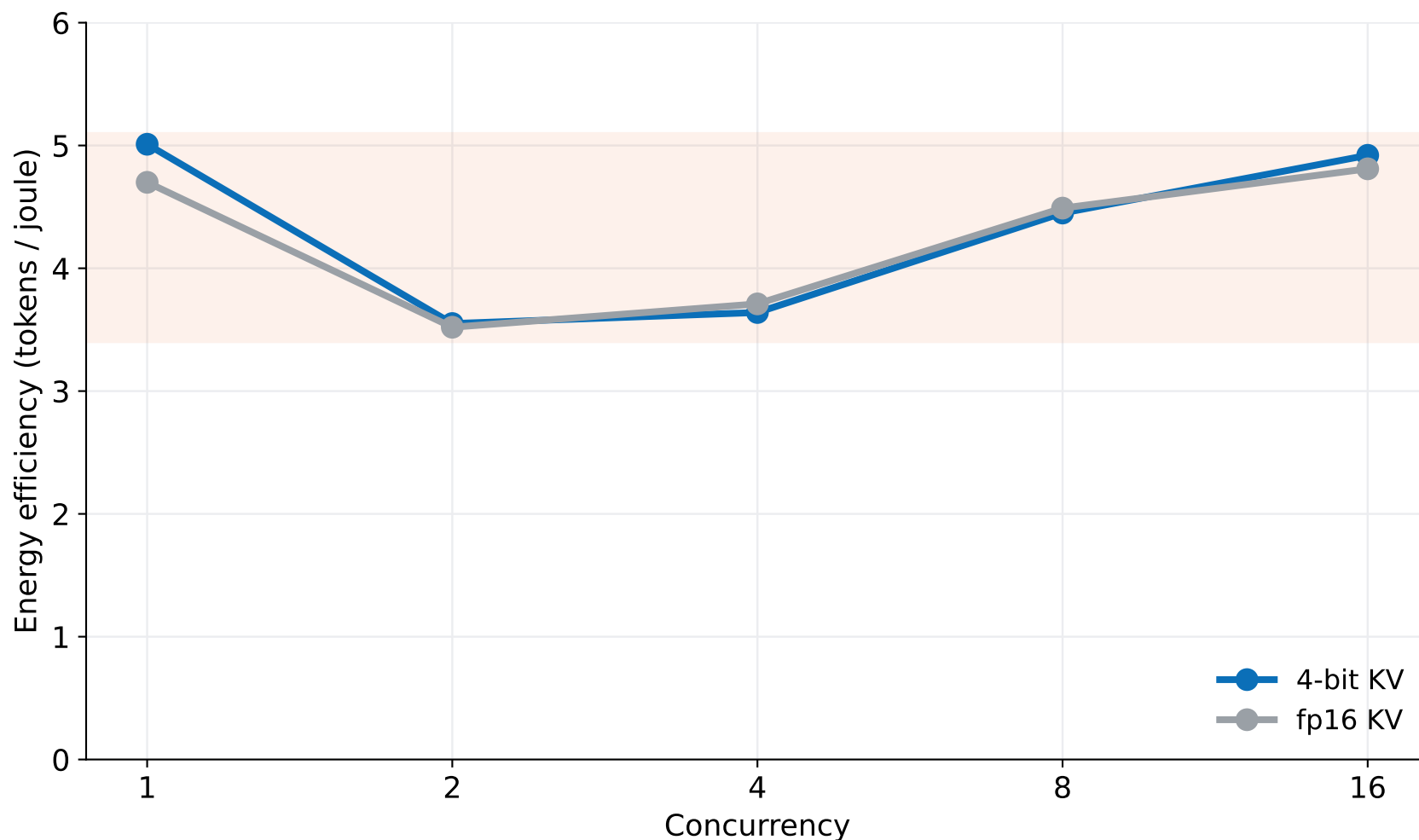
Switching the KV cache to 4-bit freed enough memory to run 8 parallel streams instead of 4 — +28% aggregate at 8 users, with no single-stream penalty.

A wider engine kills the queue



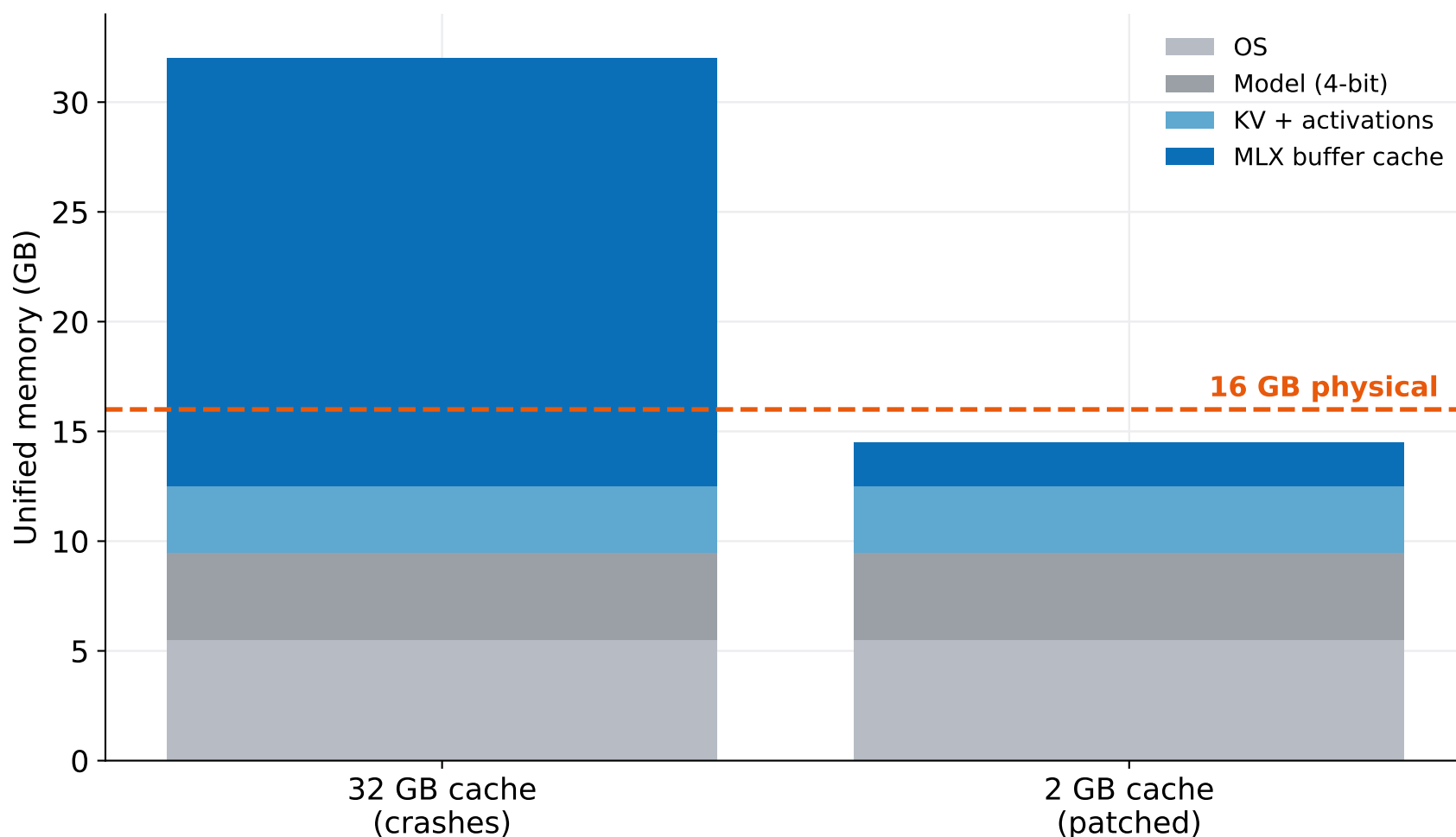
With a 4-wide engine, 4 of every 8 users wait in a queue. An 8-wide engine serves them all at once — first-token latency drops from 16.5 s to 0.58 s.

Throughput scales — efficiency stays flat



GPU power rises with concurrency in step with throughput, so tokens/joule stays flat (~4-5) across every load. Batch for throughput, not energy.

Bonus: a 32 GB cache limit that OOM'd 16 GB Macs



A hardcoded 32 GB MLX buffer-cache limit lets memory grow until a 16 GB Mac OOMs. Traced it, patched to 2 GB, and reported upstream. ► Full benchmark, scripts & live dashboard at github.com/rajatsadh24/vllm-mlx-qwen35-benchmark